

Order. Payment. Session. Attempt.

The canonical model for the payment platform: a simple client-facing surface over sophisticated internal machinery — with channel independence, and with refund and dispute modelled as money and trust objects rather than payment states.

Order / Billing → Payment → Session → Attempt
→ Decision → Execution → Money Movement
with Obligation cutting across the model

Status

Canonical — locked for the next version

New in v3.0

Refund · Dispute · Evidence graph

Client-facing objects

Order · Payment · Session

Money & Trust

Capture · Refund · Dispute · Ledger

Contents

1. The canonical philosophy	3
2. Where this started: the naming problem	4
3. The fundamental distinction	4
4. Why PaymentIntent is not the client-facing model	5
5. What happened to PaymentRequest	6
6. The canonical relationship	7
7. Why Order stays separate from Payment	8
8. Channel independence	8
9. Omnichannel, worked	9
10. Card present	11
11. Why Attempt must be separate	13
12. Refund is not a Payment state	14
13. Dispute is not a Payment state	16
14. The evidence graph	17
15. State models	19
16. Progressive disclosure	20
17. Billing and invoices	21
18. The full architecture	22
19. The eight design rules	24
20. Canonical definitions	25

The canonical philosophy

Two principles govern this architecture. Everything else follows from them.

PRINCIPLE ONE

The client-facing model should be simple. The internal platform should be extremely sophisticated.

Hide complexity from the client without removing the sophisticated payment machinery underneath.

PRINCIPLE TWO – NEW IN V2.0

A Payment is channel-independent.

A Session represents the interaction channel. An Attempt represents the execution. Card present, card not present, mobile, terminal, kiosk, wallet, payment link and agentic commerce are

1.1 The one-line model

Order / Billing → Payment → Session → Attempt
→ Decision → Execution → Money Movement

with Obligation cutting across the model wherever there are marketplaces, platforms, franchises, PayFacs, split payments, reserves, fees and payouts.

1.2 The four responsibilities

OBJECT	ANSWERS THE QUESTION	DOMAIN
Order	What does the customer owe?	Commerce
Payment	How are we collecting the money owed?	Payment — durable state machine
Session	How does the customer interact with the payment?	Interaction — presentation
Attempt	What actually happened when we tried to collect?	Execution

Each object has one clean responsibility. **Order ≠ Payment ≠ Attempt.**

1.3 What principle two buys

One architecture that naturally supports the whole span — without a separate payment model per channel:

Ecommerce → Mobile → Store → Terminal → SoftPOS → Kiosk → Endless Aisle
→ Marketplace → PayFac → Subscription → Invoice → Agentic Commerce

This is the abstraction required to be a top PSP solution provider rather than another payment gateway.

SECTION 2

Where this started: the naming problem

The original model was conceptually sound and practically confusing.

ORIGINAL MODEL	INTENDED MEANING
Order	what the customer is buying / commercial contract
↓	
PaymentIntent	durable money-collection state machine
↓	
PaymentRequest	presentation / interaction binding

THE USABILITY PROBLEM

`PaymentRequest`, `PaymentIntent` and `/payments` could **all** appear to represent "the payment." A developer reading the API could not tell which object was the money. That is a major API usability problem, and it is the reason for everything that follows.

The separation of concerns was right. The exposure of that separation to clients was wrong. This document keeps the concerns and changes the exposure.

SECTION 3

The fundamental distinction

A \$500 airline ticket, followed all the way down.

3.1 Order — what does the customer owe?

```
Order
ID:      ORD-123
Amount:  $500
Status:  OPEN
```

The Order belongs to the **commerce domain**. It answers one question: what does the customer owe?

3.2 Payment — how are we collecting it?

```
Payment
  ID:      PAY-123
  Amount:  $500
  Order:   ORD-123
  Status:  REQUIRES_PAYMENT
```

This is the **durable payment state machine**. It answers: how are we collecting the money owed by the customer?

3.3 Attempt — what actually happened?

```
Payment
├─ Attempt #1
│   └─ Visa
│       └─ Acquirer A
│           └─ Processor X
```

If the first attempt fails, the Payment does not change:

```
Payment
├─ Attempt 1 → Processor A → DECLINED
├─ Attempt 2 → Processor B → DECLINED
└─ Attempt 3 → Processor C → AUTHORIZED
```

ONE OF THE MOST IMPORTANT CONCLUSIONS IN THIS ARCHITECTURE

The Payment remains the same. Three attempts, three processors, two declines and one approval — and one durable Payment object throughout.

SECTION 4

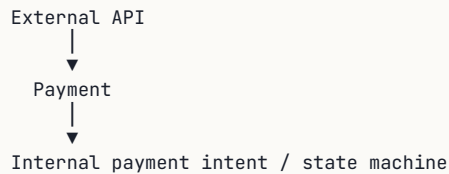
Why PaymentIntent is not the client-facing model

A sophisticated PSP needs a durable payment state machine internally. Clients should not have to understand or create it.

RESOLUTION

PaymentIntent remains a valid internal and canonical concept. The external API calls it .

You do not need to expose implementation terminology if doing so makes the platform harder to understand.



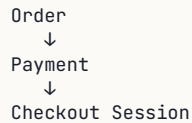
4.1 What this means in practice

```

POST /v2/orders
POST /v2/orders/{order_id}/checkout

→ checkout_url
  
```

The platform creates internally:

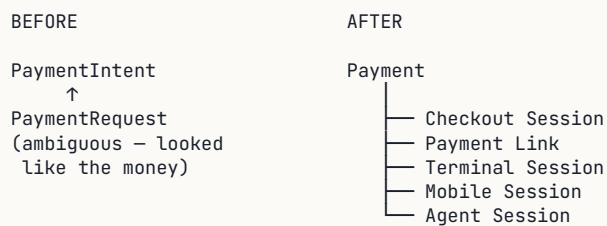


The client simply receives a `checkout_url`. This is **progressive disclosure**: the machinery exists, it is created, and the client never names it.

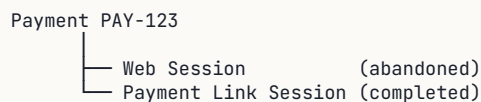
SECTION 5

What happened to PaymentRequest

It is not the durable money object. It is an interaction and presentation mechanism, and it is renamed Session.



5.1 The same payment across different channels



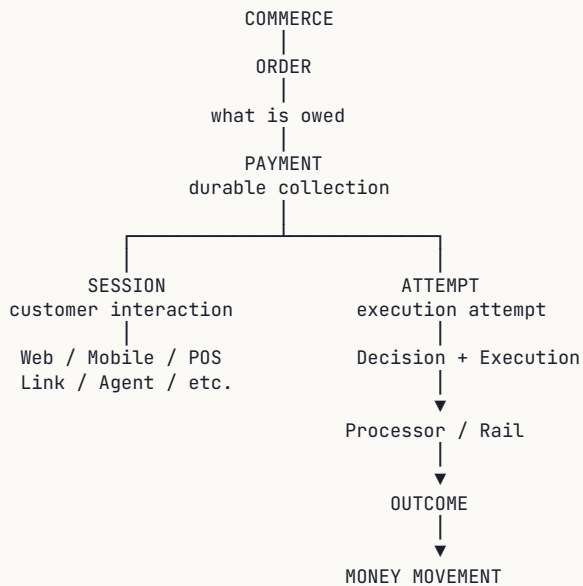
THE RULE THIS ESTABLISHES

We avoid making the presentation object responsible for the money lifecycle. A session can be abandoned, expire or be replaced without any effect on the payment it was presenting.

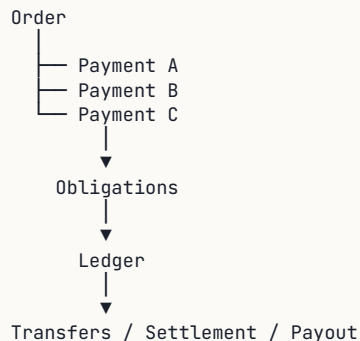
SECTION 6

The canonical relationship

The model to lock in.



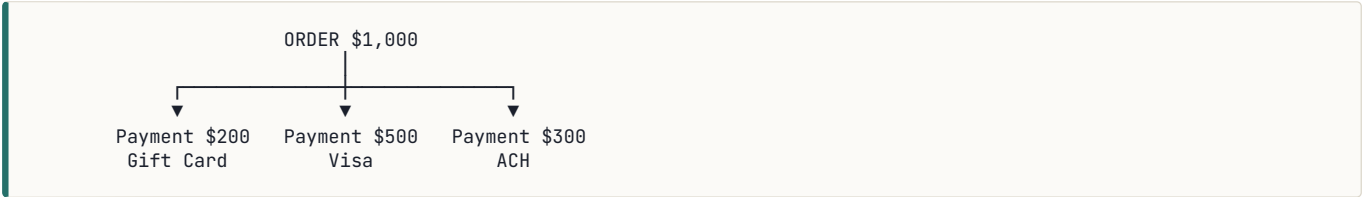
6.1 Economic allocation



SECTION 7

Why Order stays separate from Payment

A \$1,000 marketplace order, paid three ways.



RELATIONSHIP	CARDINALITY	DRIVEN BY
Order to Payment	1 : N	Split tender, deposits, multiple rails, partial collection
Payment to Session	1 : N	Channel switching, abandonment, re-presentation
Payment to Attempt	1 : N	Declines, retries, processor fallback
Order to Obligation	1 : N	Sellers, platforms, franchisors, tax authorities

THE CONCLUSION

Order ≠ Payment ≠ Attempt. Each has a clean responsibility, and none of the three can absorb another without breaking on a case the platform must support.

SECTION 8

Channel independence

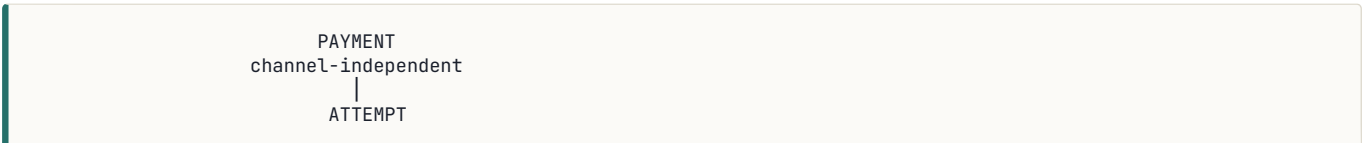
Omnichannel and card present are not edge cases to accommodate. They are the proof that separating Order, Payment, Session and Attempt is the right model.

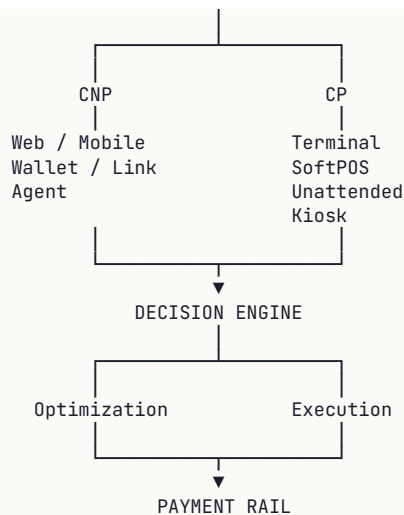
THE PRINCIPLE, STATED FOR THE WHITEPAPER

A Payment is channel-independent. A Session represents the interaction channel. An Attempt represents the execution.

Card present, card not present, mobile, terminal, kiosk, wallet, payment link and agentic commerce are execution and interaction channels — not different fundamental payment objects.

8.1 Where the channel actually lives





The channel distinction appears at the **Attempt**, not at the Payment. Both branches converge on the same Decision Engine, the same optimization, and the same rails. One architecture, many channels.

8.2 Interaction channel is not payment identity

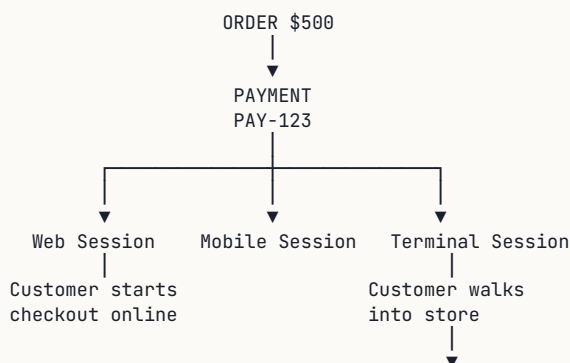
WHAT CHANGES BY CHANNEL	WHAT NEVER CHANGES
Session type and its lifecycle	The Payment and its identity
Attempt data — EMV, cryptogram, device, browser	The Order and what is owed
Entry mode and POS condition	The Decision Engine and its policies
Device and operator identification	The Money Platform and financial truth

SECTION 9

Omnichannel, worked

Three scenarios that a channel-specific payment model cannot express.

9.1 Start online, finish in store

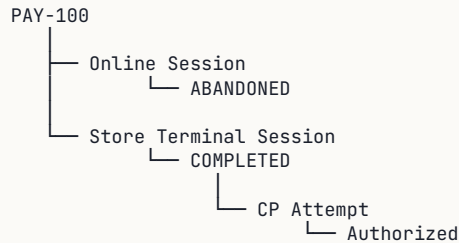




The example

A customer adds a \$500 TV online. Order = ORD-100 , Amount = \$500 , Payment = PAY-100 . The customer chooses "pay in store."

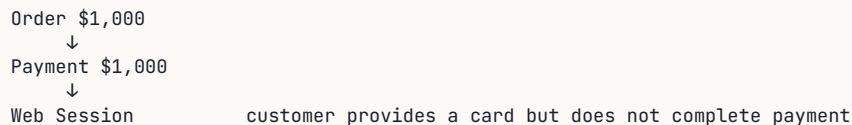
The payment is not abandoned or recreated.



The Order and Payment remain durable across channels. That is a major omnichannel capability, and it falls out of the model rather than being built as a feature.

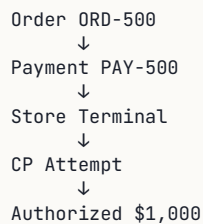
9.2 Endless aisle — a \$1,000 furniture purchase

Step 1 — online



Step 2 — store

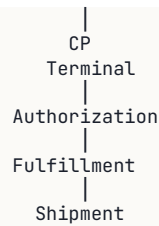
The customer visits the showroom. The associate retrieves the same order.



Step 3 — endless aisle

The store does not have the item in inventory. The associate orders it from another fulfilment location.

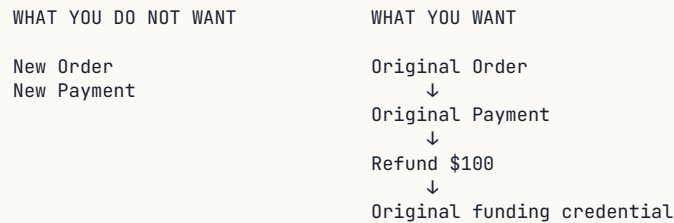




Same commercial transaction. Different interaction and fulfilment channels. **That is exactly why Order, Payment and Session should not be collapsed.**

9.3 Buy online, return in store

An order of \$300, paid \$300 through e-commerce. The customer later walks into a physical store and returns \$100 of it.



The refund originates from a **card-present servicing interaction**, while the original payment was card-not-present.

THE RESTATEMENT

Interaction channel ≠ payment identity. A refund initiated at a terminal against a payment taken online is one payment, serviced through a second channel — not a new payment object.

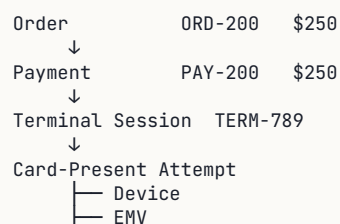
SECTION 10

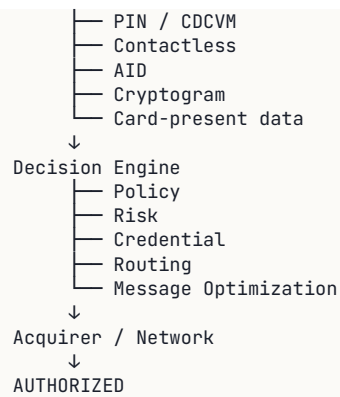
Card present

A pure in-store transaction, and the device platform it requires.

10.1 The transaction

A customer walks into a store and purchases \$250 of merchandise.





THE IMPORTANT THING

Card present is an execution channel — not a different Payment model. The Order, the Payment and the Decision Engine are identical to the e-commerce case. What differs is the Session type and the data carried on the Attempt.

10.2 But card present needs a device platform

Beneath the common payment model, card present requires its own platform capabilities. These attach to the device and the store, not to the Payment.

CAPABILITY	
Terminal and device identity	Store and location
Device provisioning	Merchant and operator identification
Remote configuration	Tipping
EMV configuration	Incremental authorization
Key management and injection	Partial approval
Contact and contactless	Void
PIN	Card-present refunds
Offline authorization controls	Device health
Network connectivity management	Software and firmware lifecycle
Offline and stand-in behaviour	

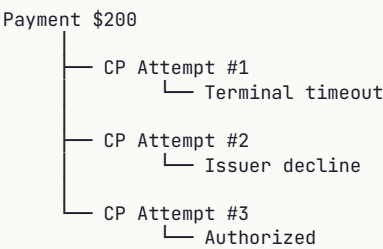
WHY THIS MATTERS STRUCTURALLY

This is why card present must **not** be forced into a web-style Checkout Session. The Session abstraction is correct — a terminal session is a session — but the device platform beneath it is specialised and substantial, and it has no equivalent on the web side.

Why Attempt must be separate

Card present demonstrates this more sharply than any other channel.

Consider a \$200 terminal transaction.



THE REQUIREMENT

The customer should not accidentally get three \$200 charges.

The Payment is the durable business and payment object. The Attempts are executions against that payment.

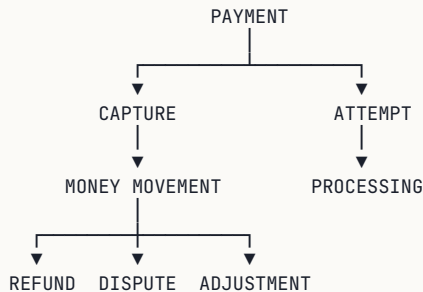
11.1 What this distinction makes possible

CAPABILITY	WHY IT DEPENDS ON ATTEMPT BEING SEPARATE
Terminal retries	A retry is a new Attempt, not a new Payment. The cashier presses again and nothing duplicates.
Duplicate prevention	One durable object to check against. Duplicate detection becomes structural rather than forensic.
Offline transactions	A stored offline transaction forwards later as an Attempt under the Payment it always belonged to.
Network failures	A timeout is an Attempt outcome, not an unknown state on the money object.
Processor failover	Attempt two routes elsewhere. The Payment does not change and the merchant sees no churn.
Partial approvals	The approved amount attaches to the Attempt; the Payment still knows what it set out to collect.
Reversals	Reverse a specific Attempt without disturbing the Payment's pursuit of the outcome.
Late responses	A response arriving after a timeout resolves against the Attempt that produced it.
Idempotency	The Payment is the anchor. Every retry keys against it rather than against an invented client-side key.

Refund is not a Payment state

A refund is a money movement associated with a payment. It belongs to the Money Platform, not to the payment lifecycle.

12.1 Where refund sits



A refund is the merchant saying: **"return some or all of the money from this Payment to the customer."** It does not change what the Payment set out to do.

12.2 Worked example — a \$500 TV

Order	\$500		
Payment	\$500		
Authorization	\$500		
Capture	\$500		
Customer returns \$150 of merchandise:			
Refund \$150	→	Captured	\$500
		Refunded	\$150
		Net	\$350
A further \$350 refund:			
Refund \$350	→	Refunded	\$500
		Net	\$0

WHAT WE DO NOT DO

We do not make the Payment state simply become `REFUNDED`. That would collapse a money movement into a lifecycle state and lose the ability to express partial and multiple refunds.

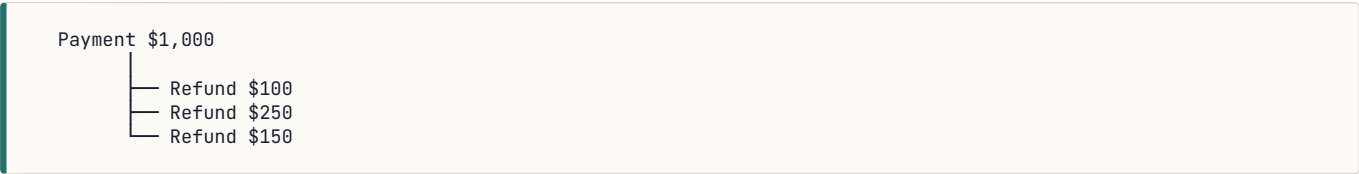
What we expose instead

```
Payment
  status:          CAPTURED
  refunds:
    - $150
    - $350
  refunded_amount: $500
  remaining_amount: $0
```

A customer-facing aggregate status such as `REFUNDED` may be **derived** from these fields. Internally the Payment lifecycle remains clean.

12.3 Why refund must be a separate object

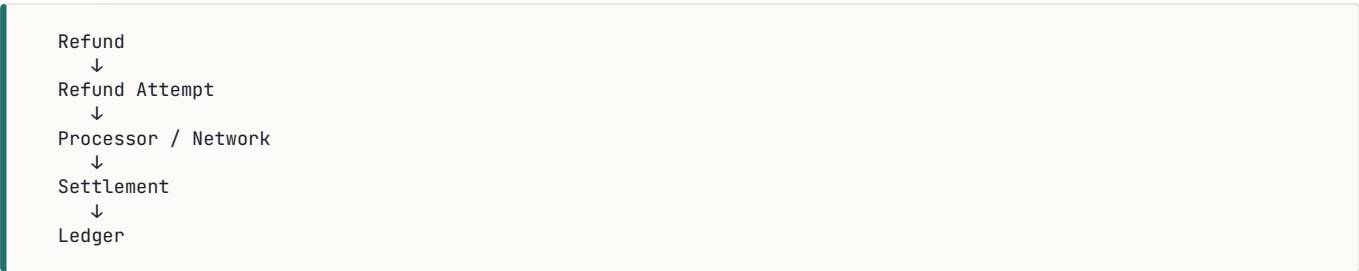
Partial refunds make this obvious.



Independent objects are required for each of the following, none of which can be represented by a status field on the Payment:

NEEDS ITS OWN OBJECT	NEEDS ITS OWN OBJECT
Partial refund	Refund attempt
Multiple refunds	Refund timing
Refund reason	Refund failure
Refund destination	Refund reconciliation
Refund status	

12.4 Refund reuses the execution architecture



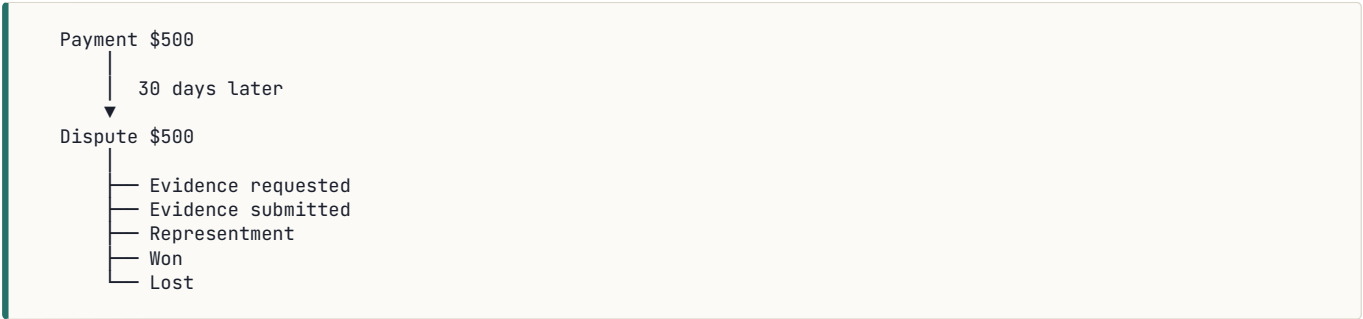
THE CONSEQUENCE WORTH NOTICING

A refund has attempts for the same reasons a payment does — it can fail, time out, or need a different route. **The same Decision and Execution architecture can therefore be used for refunds.** One execution engine serves both directions of money movement.

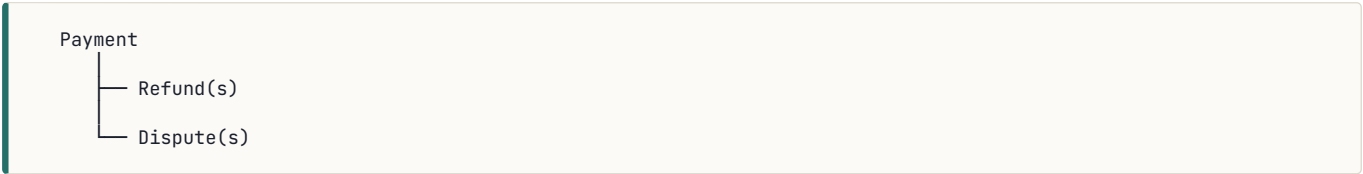
Dispute is not a Payment state

A dispute is not initiated by the merchant. It is an external event arriving through the issuer and card-network ecosystem.

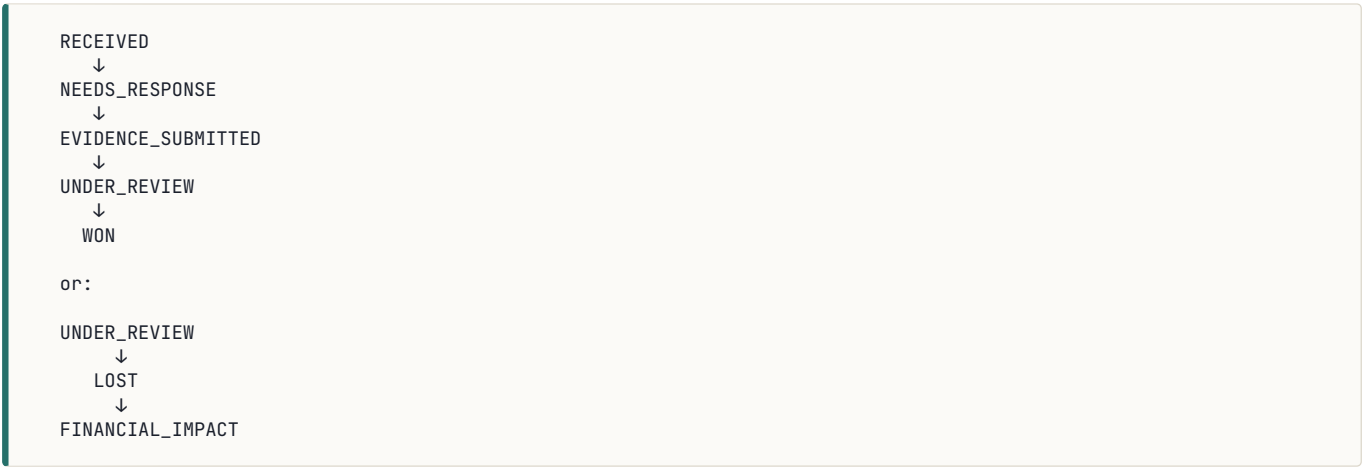
13.1 The shape of a dispute



Dispute is therefore a **first-class Money and Trust object associated with a Payment** — not a Payment state.



13.2 Dispute has its own lifecycle



And the wider set of processes it must accommodate

PROCESS	PROCESS
Pre-dispute alerts	Representation
Inquiry	Second presentment

PROCESS	PROCESS
Retrieval	Arbitration
Chargeback	Good-faith processes

THE SAME PATTERN AS RAILS

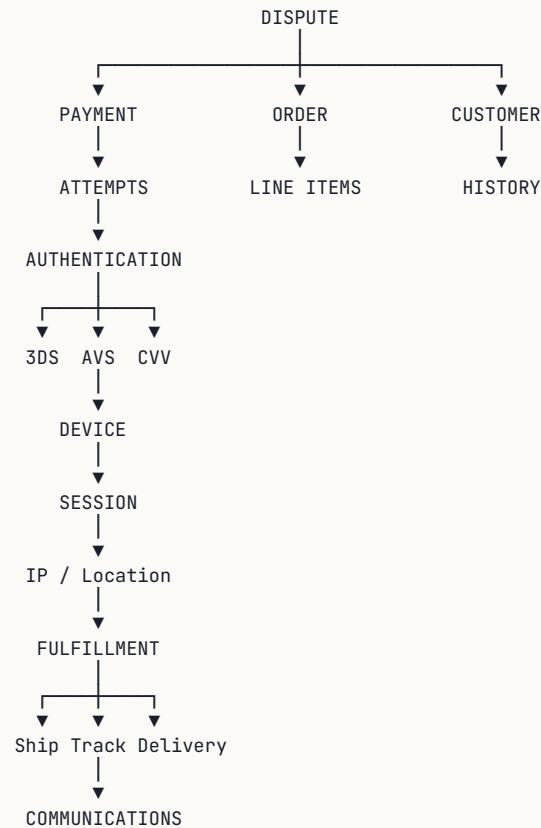
The exact lifecycle varies by rail and network. This is another place where **a common abstraction with rail-specific lifecycles beneath it** is required — the same structural answer used for payment rails in Section 8.

SECTION 14

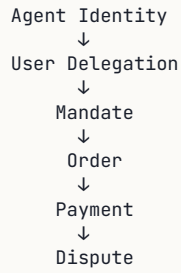
The evidence graph

This is where the platform differentiates. A dispute should not simply point at a payment.

14.1 The graph



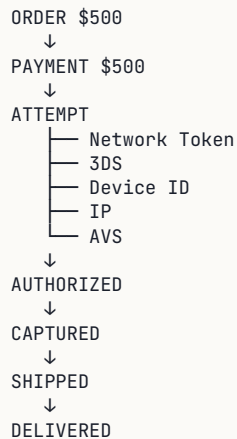
14.2 The agentic extension



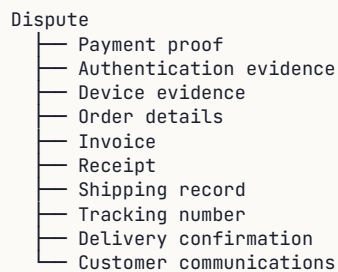
For an agentic transaction the chain of authority becomes part of the evidence. That is a very powerful dispute evidence platform, and it is only possible because the objects were kept separate in the first place.

14.3 Worked example — an e-commerce dispute

A customer buys a \$500 laptop online.



Sixty days later the customer claims the merchandise was not received. The dispute platform automatically assembles:



An evidence and decision engine then determines what should be submitted.

WHY THIS IS THE DIFFERENTIATOR

That is much more powerful than simply storing a dispute status.

Every element of the graph already exists as a separate object in this architecture. A platform that collapsed attempt into payment, or session into payment, could not assemble it — the evidence would have been overwritten by the next state transition.

SECTION 15

State models

Three independent state machines. None inherits another's state.

15.1 Order

```
DRAFT
↓
OPEN
↓
FULFILLING
↓
FULFILLED
↓
CLOSED
```

Side states: CANCELLED EXPIRED RETURNED

Order status answers: **what is happening with the commercial transaction?**

15.2 Payment

WHY THERE IS NO SUCCEEDED STATE

A generic `SUCCEEDED` hides an important distinction. It does not say whether the money has actually moved. The state model therefore names each stage explicitly.

```
REQUIRES_PAYMENT
↓
REQUIRES_ACTION
↓
PROCESSING
↓
AUTHORIZED
↓
PARTIALLY_CAPTURED
↓
CAPTURED
↓
SETTLED
```

with failure, cancellation and expiration paths

THE CRITICAL DISTINCTION

AUTHORIZED ≠ CAPTURED ≠ SETTLED

A hotel authorizes \$500 today — the Payment is AUTHORIZED. It captures when the guest checks out — the Payment is CAPTURED. The money settles later — the Payment is SETTLED. That distinction is essential for a serious PSP.

15.3 Session

A Checkout Session does not inherit the Payment state.

```
CREATED
↓
ACTIVE
↓
IN_PROGRESS
↓
COMPLETED

Side states:  ABANDONED  EXPIRED  CANCELLED
```

The example that makes the separation clear

```
Customer completes checkout:

    Session = COMPLETED
    Payment = AUTHORIZED

That is perfectly valid.
```

The session being completed does not mean the money is captured or settled. **This separation is extremely important** — and Section 9.1 shows the other half of it, where an ABANDONED session leaves the Payment entirely intact.

SECTION 16

Progressive disclosure

The most important API simplification. The enterprise client can see the sophisticated platform without forcing the SMB to learn it.

Level 1 — SMB and simple checkout

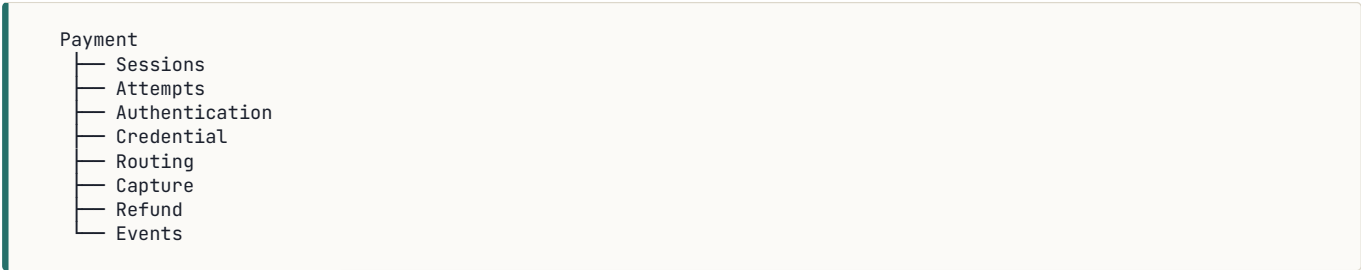
```
POST /v2/orders
POST /v2/orders/{id}/checkout
```

The platform creates Order, Payment and Session. The merchant receives a checkout URL.

Level 2 — Enterprise e-commerce

```
POST /v2/payments
POST /v2/payments/{id}/confirm
POST /v2/payments/{id}/capture      (optional)
```

Level 3 — Advanced

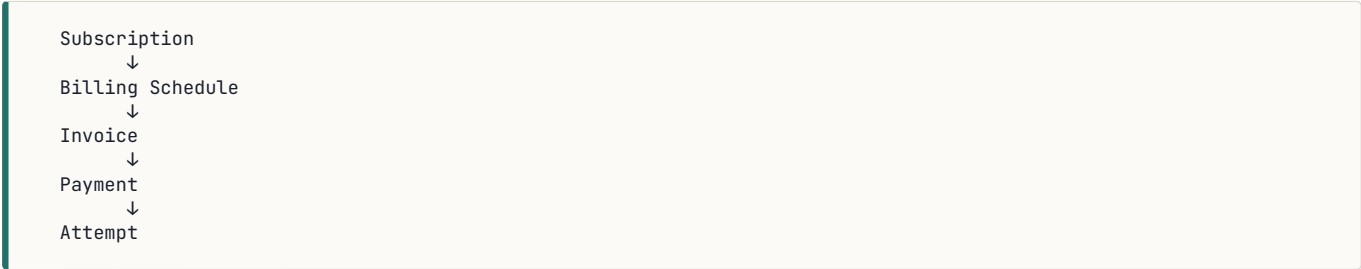


CLIENT	WHAT THEY INTERACT WITH	WHAT THE PLATFORM STILL DOES
SMB	Order, checkout URL	Creates Payment, Session and Attempts; runs the full decision engine
Enterprise e-commerce	Payment, confirm, capture	Creates Sessions and Attempts; routing, retry and recovery
Omnichannel retail	Payment plus multiple Sessions	Maintains payment identity across web, mobile and terminal
Advanced	The whole lifecycle	Executes the plan the client can now observe

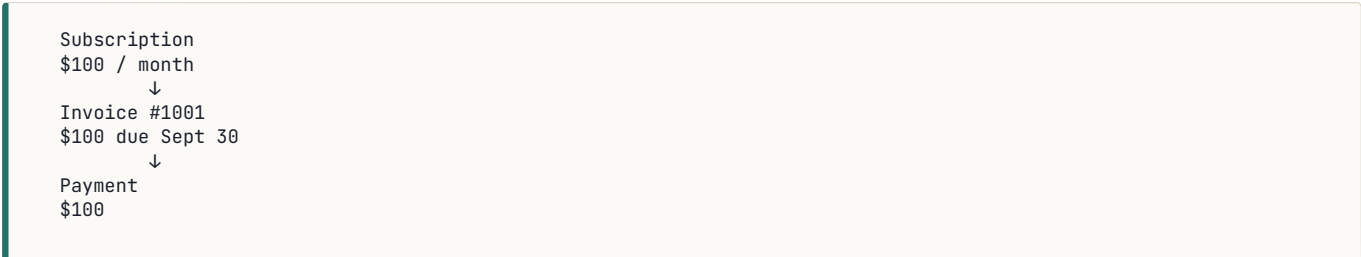
SECTION 17

Billing and invoices

An invoice must not become another name for Payment.



17.1 Worked example



↓
Attempt #1 Network Token → Acquirer A → Declined
↓
Attempt #2 Network Token → Acquirer B → Authorized

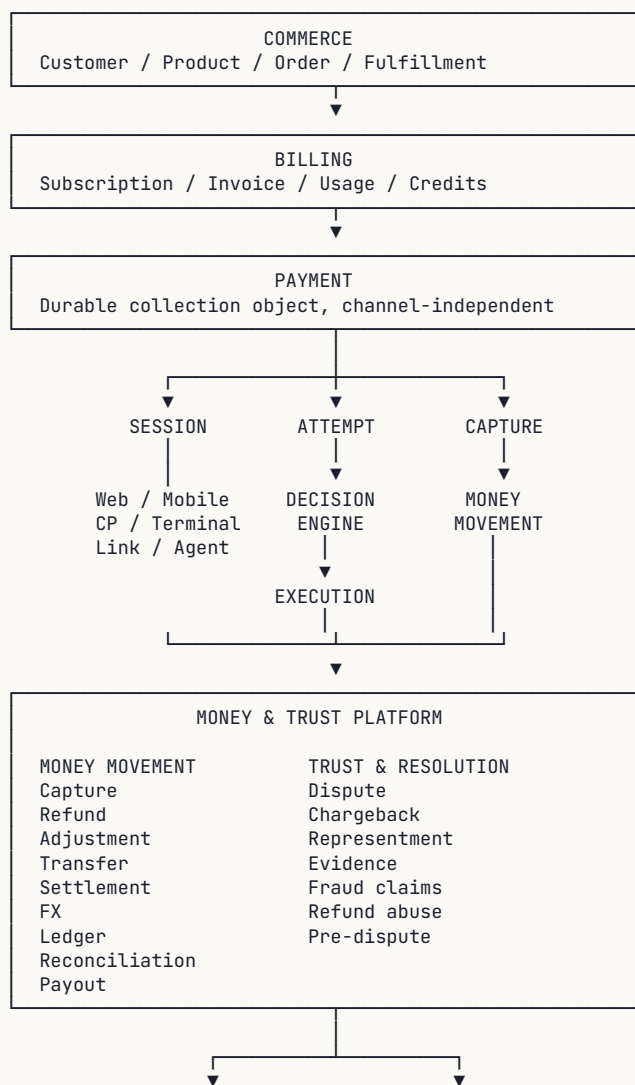
THE PRINCIPLE, RESTATED

The **Invoice is what is owed**. The **Payment is the mechanism for collecting it**. This is the same relationship Order has to Payment, which is why one Payment object serves both commerce and billing without a second implementation.

SECTION 18

The full architecture

The complete stack, with channels expressed between Payment and Attempt.



REFUND

DISPUTE

EVIDENCE / RESOLUTION

18.1 What changed across versions

CHANGE	REASON
Channels expressed explicitly between Payment and Attempt	Makes channel independence visible in the diagram rather than implied by it.
Commerce layer gains Fulfillment	Endless aisle and ship-from-store make fulfilment a first-class commerce concern.
Billing layer gains Credits	Completes the billing domain alongside subscription, invoice and usage.
Execution layer names the entry modes	CNP, CP, SoftPOS, terminal, kiosk and offline are execution characteristics, shown where they belong.
v3.0 — Capture promoted beside Session and Attempt	Capture is a money movement against a Payment, not a Payment state. Showing it as a sibling makes that explicit.
v3.0 — Money Platform renamed Money & Trust Platform	It contains two related but different domains: money movement, and trust and resolution. Section 18.2.
v3.0 — Refund and Dispute shown as outputs of the platform	Neither is a Payment state. Both are associated objects with their own lifecycles.

18.2 Why "Money & Trust Platform"

The bottom layer contains two related but different domains, and naming only one of them understated half the platform.

MONEY MOVEMENT	TRUST & RESOLUTION
Capture	Dispute
Refund	Chargeback
Adjustment	Representment
Transfer	Evidence
Settlement	Fraud claims
FX	Refund abuse
Ledger	Pre-dispute
Reconciliation	
Payout	

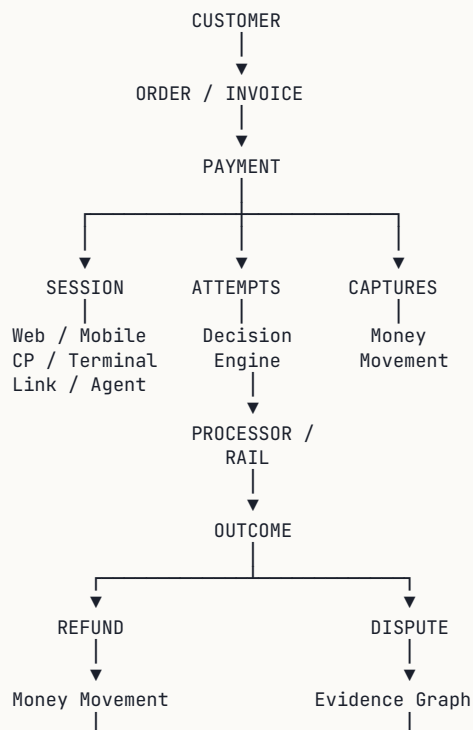
That gives a more complete PSP platform than money movement alone.

The eight design rules

The separations that make the model work, stated as rules a reviewer can check an implementation against.

#	RULE	WHY
1	Order is not Payment	Order describes the commercial transaction. One order may carry many payments, and may carry none.
2	Invoice is not Payment	Invoice describes money due. Payment is the mechanism for collecting it.
3	Payment is not Attempt	Payment is durable; attempts are executions. Three attempts must never become three charges.
4	Session is not Payment	Session represents the interaction channel. It can be abandoned without touching the money.
5	Capture is not Payment	Capture is a money movement against a payment, and there may be several.
6	Refund is not a Payment state	Refund is a money movement associated with a payment. Partial and multiple refunds make a status field untenable.
7	Dispute is not a Payment state	Dispute is a resolution and claims lifecycle associated with a payment and its broader evidence graph, initiated externally.
8	Ledger is the financial truth	Payment status describes the payment lifecycle. The ledger describes what actually happened financially.

19.1 The final object relationship



LEDGER /
RECONCILIATION

WHAT THE EIGHT RULES BUY

One core payment architecture for every use case.

E-commerce, card present, omnichannel, subscriptions, invoices, marketplaces, PayFacs, AFT and OCT, bank rails and agentic commerce — without creating a different core payment architecture for each.

SECTION 20

Canonical definitions

The definitions to be used consistently across every document, API surface and conversation from this point.

Order

What the customer bought, and what is commercially owed.

Belongs to the commerce domain. Its lifecycle is driven by the commercial transaction, not by money movement.

Billing

When and why money becomes due.

Subscription, billing schedule, invoice, usage and credits. Produces an amount owed on a schedule, which a Payment then collects.

Payment

The durable commitment to collect that money.

The durable state machine, and **channel-independent**. Survives declines, retries and channel changes. Exposed to clients as `Payment`; known internally as the payment intent.

Session

How the customer interacts with the payment.

Checkout, payment link, terminal, mobile, agent. Represents the interaction channel. Has its own lifecycle and never determines whether money moved.

Attempt

A specific execution of the payment.

One try against one processor with one credential. Carries the execution characteristics — CNP or CP, entry mode, EMV and device data. Many attempts may belong to one Payment.

Decision Engine

Determines the best way to execute the attempt.

Policy, screening, risk, authentication, credential decisioning, routing, message optimization, retry and recovery. Shared by every channel.

Orchestrator

Executes the selected plan.

Carries out what the Decision Engine determined, against the chosen processor and rail.

Capture

A money movement against a payment.

Not a payment state and not necessarily singular. Several partial captures may belong to one Payment.

Refund

Returning some or all of the money from a payment to the customer.

A money movement associated with a Payment, with its own reason, destination, status, timing and attempts. Never a Payment state. A customer-facing `REFUNDED` status may be derived from refunded and remaining amounts.

Dispute

A claim against a payment, initiated externally.

Arrives through the issuer and card-network ecosystem. A first-class Money and Trust object with its own lifecycle — inquiry, retrieval, chargeback, representment, second presentment, arbitration. Never a Payment state.

Evidence Graph

The complete transaction context assembled for a dispute.

Payment, order, customer, attempts, authentication, device, session, location, fulfilment and communications — plus agent identity, delegation and mandate for agentic transactions.

Money & Trust Platform

Determines the financial truth, and resolves claims against it.

Money movement — capture, refund, adjustment, transfer, settlement, FX, ledger, reconciliation, payout. Trust and resolution — dispute, chargeback, representment, evidence, fraud claims, refund abuse, pre-dispute. Financial truth lives here, not in a payment status field.

Obligation

Who ultimately has an economic claim on the money.

Cuts across the model. Applies to marketplaces, platforms, franchises, PayFacs, split payments, reserves, fees and payouts.

Device Platform

What card-present acceptance requires beneath the Session.

Device identity, provisioning, remote configuration, EMV parameters, key injection, offline controls, health and firmware lifecycle. Specialised to card present; attaches to the device and store rather than to the Payment.

THE MODEL, LOCKED

Order / Billing → Payment → Session → Attempt → Decision → Execution → Money Movement

A Payment is channel-independent. A Session represents the interaction channel. An Attempt represents the execution. One architecture supports ecommerce, mobile, store, terminal, SoftPOS, kiosk, endless aisle, marketplace, PayFac, subscription, invoice and agentic commerce — without creating separate payment models for every channel.